Template Method

Template Method · Mine CSV/JSON reports with shared pipeline · Build with Waqa

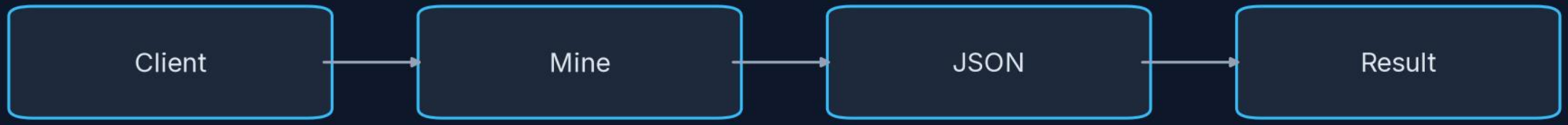
Today's Hook

- Problem: Mine CSV/JSON reports with shared pipeline
- Pattern: Template Method — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Mine CSV/JSON reports with shared pipeline
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Template Method: Mine CSV/JSON reports with shared pipeline

Real-World Scenario

- Running example: Mine CSV/JSON reports with shared pipeline
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.templatemethod.problem;

import java.util.*;

public final class NaiveMine {
  public static String csv(String raw){ return "rows=" + raw.split("\n").length; }
  public static String json(String raw){ return "rows=" + raw.split("\\{", -1).length; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.behavioral.templatemethod.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.*;

abstract class DataMiner {
  final String mine(String raw){ String opened=open(raw); List<String> rows=parse(opened); return analyze(rows); }
  abstract String open(String raw); abstract List<String> parse(String opened);
  String analyze(List<String> rows){ return "rows="+rows.size(); }
}
final class CsvMiner extends DataMiner {
  String open(String raw){ return raw; }
  List<String> parse(String opened){ return Arrays.stream(opened.split("\n")).filter(s->!s.isBlank()).toList(); }
}
final class JsonMiner extends DataMiner {
  String open(String raw){ return raw; }
  List<String> parse(String opened){ return Arrays.stream(opened.split("\n")).filter(s->s.contains(":")).toList(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.templatemethod.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Template Method: CSV/JSON mine ===");
    System.out.println("--- naive ---"); System.out.println(NaiveMine.csv("a\nb"));
System.out.println("--- template ---");
System.out.println(new CsvMiner().mine("a\nb\n"));
System.out.println(new JsonMiner().mine("{\n x:1\n}\n"));
  }
}
```

Walkthrough Output

```
=== Template Method: CSV/JSON mine ===
--- naive ---
rows=2
--- template ---
rows=2
rows=1
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Template Method.

Related Patterns

- Contrast Template Method with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Template Method → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.